

Conditional Jacobian Aggregation

Optimizing GPU Track Finding through Algorithmic Refinement



Hao-Chun, Liang

GPU Optimization for ACTS/traccc

Advisor: Prof. Bo-Cheng, Lai

January 2026

- 1 Problem Context
- 2 Implementation
- 3 Results
- 4 Lessons Learned
- 5 Conclusion

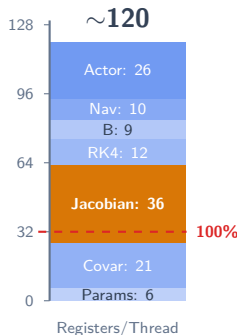
The Problem

- `propagate_to_next_surface` kernel
- Only **10-25% GPU occupancy**
- Root cause: High register usage
- V100 needs ≤ 32 regs for 100% occupancy

Impact

- SM underutilization
- Poor latency hiding
- Reduced throughput

Register Budget Breakdown



Key Insight

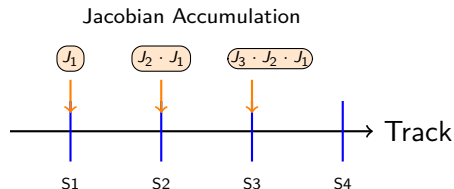
Jacobian only needed for MBF smoother!

When is Jacobian Needed?

- Multi-Branch Fit (MBF) smoother requires accumulated Jacobian
- Used for backward smoothing pass
- When MBF disabled: Jacobian computed but **never used**

Current Behavior (parameter_transporter)

- 1 Compute full 6×6 Jacobian at each surface
- 2 Multiply: $J_{acc} = J_{new} \times J_{acc}$ ($6 \times 6 \times 6 \times 6$)
- 3 Store accumulated Jacobian to global memory
- 4 **Wasted work when MBF=false!**



Savings per Surface

- ~ 216 FLOPs (matrix mult)
- ~ 288 bytes memory I/O

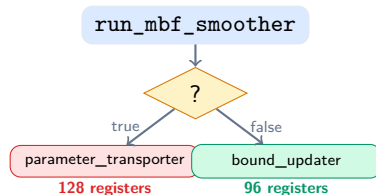
Solution: Two Actor Chains

Original: `ckf_actor_chain_t`

- 1 `pathlimit_aborter`
- 2 `parameter_transporter` ← Jacobian ptr
- 3 `interaction_register`
- 4 `ckf_interactor`
- 5 `parameter_resetter`
- 6 `momentum_aborter`
- 7 `ckf_aborter`

New: `ckf_actor_chain_no_mbf_t`

- 1 `pathlimit_aborter`
- 2 `bound_updater` ← **Empty state!**
- 3 `interaction_register`
- 4 `ckf_interactor`
- 5 `parameter_resetter`
- 6 `momentum_aborter`
- 7 `ckf_aborter`



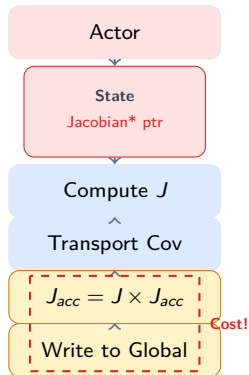
Compile-Time Dispatch

if constexpr selects actor chain based on propagator type

Key Code: Actor State Comparison

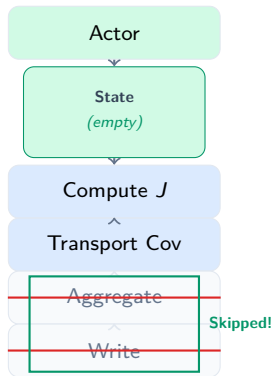
parameter_transporter

(detray - original)



bound_updater

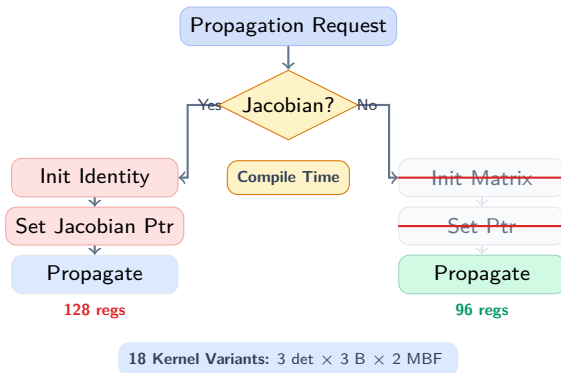
(traccc - optimized)



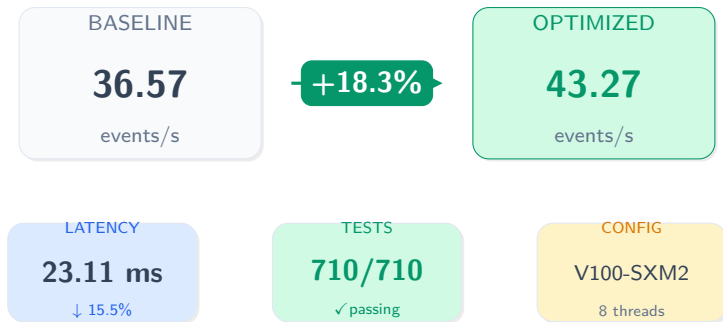
Key Difference

Empty state → **skip 6×6 matrix mult & global memory write**

Kernel Dispatch Logic



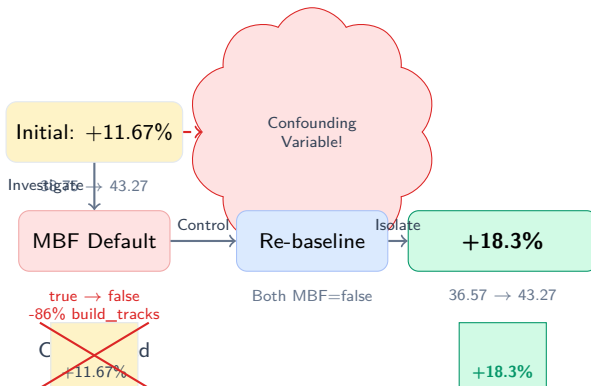
Benchmark Results



Apples-to-apples: Both with MBF=false

Dataset: ttbar_mu200

Methodology: Isolating the True Improvement



Lesson Learned

Always control for configuration changes! The true optimization impact was **larger** than initially measured.

NCU Profiling: Key Metrics Dashboard

RTX 2080 Ti (sm_75), ncu 2024.1.1

REGISTERS

128 → 96

↓ 25%

OCCUPANCY

48.6%

↑ 9.3%

INSTRUCTIONS

78.5M

↓ 4.6%

DURATION

3.61 ms

↓ 9.5%

Dual Optimization Mechanism

Register Reduction

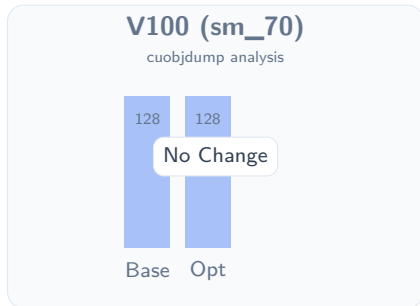
→ Higher occupancy

Instruction Reduction

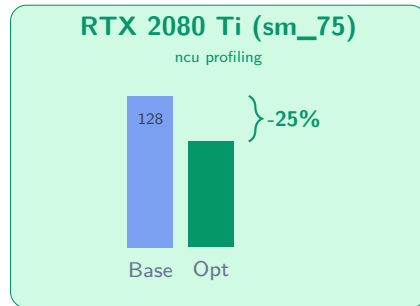
→ Less compute work

Combined: 9.5% kernel speedup

Architecture-Dependent Behavior



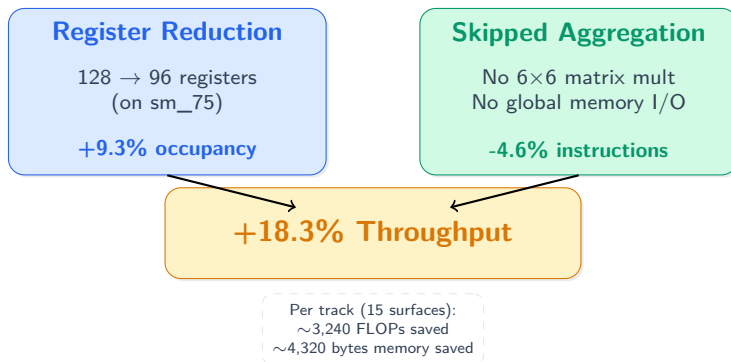
Different
→
Compiler



Key Insight

Always profile on target hardware! The same code compiles to different register counts on different architectures.

Two Complementary Mechanisms



Key Takeaways: Paper Analysis vs Real Hardware

Paper Analysis
(Theoretical estimates)

Registers:	-36	≈
Occupancy:	+10-25%	≈
Throughput:	+5-15%	✓



Profile!



Real Hardware
(NCU profiling, sm_75)

Registers:	-32
Occupancy:	+9.3%
Throughput:	+18.3%



Important Caveat

Optimization only works when **MBF smoother is disabled**. Trade-off: No backward smoothing.

Achievements

- **+18.3%** throughput improvement
- **-25%** register usage (on sm_75)
- **+9.3%** achieved occupancy
- **710/710** tests passing

Implementation

- New `bound_updater` actor
- Two actor chain variants
- 18 kernel specializations
- Compile-time dispatch

Mechanism

- Skip 6×6 matrix multiplication
- Reduce global memory traffic
- Architecture-dependent register savings

Key Files

- `bound_updater.hpp`
- `combinatorial_kalman_filter_types.hpp`
- `propagate_to_next_surface.ipp`

Conclusion

Conditional Jacobian Aggregation provides **+18.3% throughput** through dual mechanism: register reduction (architecture-dependent) and skipped matrix operations (universal).

Thank you for your attention!

+18.3%

Throughput

-25%

Registers

710/710

Tests Passing

`github.com/acts-project/tracc`

Documentation: `doc/conditional_jacobian_transport_*.md`

Q1: Why does Jacobian computation still happen?

- `get_full_jacobian()` is needed for covariance transport
- We skip the *aggregation*, not the computation
- Future work: skip computation entirely when not needed

Q2: Why not runtime dispatch instead of compile-time?

- `if constexpr` allows dead code elimination
- Runtime `if` would still allocate registers for Jacobian pointer
- Compile-time dispatch enables different kernel specializations

Q3: What is MBF smoother and when is it needed?

- Multi-Branch Fit smoother: backward pass to improve track parameters
- Needed for high-precision physics measurements
- Can be disabled for real-time trigger applications

Q4: Why different results on sm_70 vs sm_75?

- CUDA compiler makes architecture-specific optimizations
- Different instruction sets (sm_75 has tensor cores)
- Register allocation heuristics vary by architecture
- Always profile on production hardware

Q5: What about other register-heavy kernels?

- `fit_forward`: 168 registers
- `fit_backward`: 168 registers
- Similar conditional optimization could apply
- Future work opportunity

Q6: How does this compare to other optimizations?

- Multi-event batching: +93% throughput
- Conditional Jacobian: +18.3% throughput
- Complementary optimizations, can be combined

Full Profiling Data (RTX 2080 Ti, sm_75, ncu 2024.1.1)

Metric	Base	Opt	Δ
<i>Occupancy</i>			
Registers/thread	128	96	-25%
Block limit	4	5	+25%
Theor. occupancy	50.0%	62.5%	+12.5%
Achieved occupancy	39.28%	48.62%	+9.34%
<i>Compute</i>			
Duration (ms)	3.99	3.61	-9.5%
Instructions (M)	82.3	78.5	-4.6%
IPC	0.26	0.26	0%

Metric	Base	Opt	Δ
<i>Memory</i>			
Mem throughput	218 GB/s	245 GB/s	+12%
DRAM throughput	33.2%	37.7%	+4.5%
L1 hit rate	54.1%	46.3%	-7.8%
L2 hit rate	79.6%	76.6%	-3.0%
<i>Scheduler</i>			
No eligible (%)	93.24%	93.27%	+0.03%
Active warps/sched	3.23	4.03	+0.80
Warp cycles/instr	47.74	59.92	+25%

Bottleneck Analysis

Kernel remains **latency-bound**: 93% of cycles have no eligible warps. Note: Warp cycles/instruction *increased* despite speedup—fewer total instructions but each waits longer (more memory parallelism).